Practice 3 - Visual Servoing

Robotics and Cyber-Physical Systems
School of Sciences and Engineering
Tecnológico de Monterrey

Introduction

This practice introduces visual servoing—the technique of using camera feedback to control robot motion. You will learn how to process camera images, detect objects using color segmentation, compute visual error signals, and implement closed-loop control that moves the robot to center objects in its field of view.

The practice is structured in five progressive parts: first you'll add manual control to the boilerplate, then display camera feeds (RGB and depth), create a color mask visualization, implement proportional visual servoing with manual override, and finally integrate depth measurements. By the end, you'll have a complete system that tracks colored objects while allowing operator intervention and displaying real-time distance information.

Each part builds on the previous one, developing a complete understanding of vision-based robot control through the `stretch_toolkit` interface.

Before You Begin

If you previously cloned this repository, update it to the latest version to ensure you have all the necessary files and recent improvements. **Important:** The update command below will overwrite any local changes you've made. If you have files you want to keep (such as solutions to previous activities), make a backup copy outside the repository folder before proceeding.

To update your repository:

```
git pull
git fetch origin
git reset --hard origin/main
```

If dependencies were updated, synchronize your environment:

```
uv sync
```

Part 1 - Manual Control Foundation

Before implementing visual servoing, you need a working control system. In this part, you'll add manual control to the boilerplate so you can position the robot using a gamepad or keyboard.

Objective

Add teleoperation capabilities to `stretch_boilerplate.py`, enabling manual control of the robot's head and base. This provides the foundation for later parts where you'll blend autonomous visual servoing with operator override.

Task

Copy `stretch_boilerplate.py` to a new file called `p3_1_manual.py`.

In the boilerplate, the main loop currently has a placeholder for your logic:

```
while True:
    # --- Loop setup ---
    t = controller.get_time()
    velocities = {}

    # --- Your logic here ---

    # --- Send commands ---
    controller.set_velocities(velocities)
    cv2.waitKey(1)
    time.sleep(1 / 30)  # 30 Hz
```

The `teleop` object provides a method `get_normalized_velocities()` that returns a dictionary of joint velocities based on the current state of your gamepad or keyboard. Retrieve these velocities and assign them to the `velocities` variable.

After your changes, the loop should look like this:

```
while True:
    # --- Loop setup ---
    t = controller.get_time()
    velocities = {}

    # --- Your logic here ---
    velocities = teleop.get_normalized_velocities()

    # --- Send commands ---
    controller.set_velocities(velocities)
    cv2.waitKey(1)
    time.sleep(1 / 30)  # 30 Hz
```

Expected Result

Run your script and verify that you can manually control the robot:

```
uv run p3_1_manual.py
```

You should be able to:

- Drive the base forward/backward and rotate left/right
- Pan and tilt the head camera

- Extend and retract the arm

Note: The boilerplate already imports `teleop` and sets up the control loop structure. Your task is to connect the teleop input to the controller output with a single line of code.

Part 2 - Camera Feed Display

Visual servoing requires camera input. In this part, you'll display both RGB and depth camera feeds from the robot's head camera, maintaining manual control from Part 1.

Objective

Extend `p3_1_manual.py` to display live camera feeds in separate windows. This allows you to see what the robot sees while controlling it manually—essential for understanding the visual feedback used in later parts.

Task

Copy `p3_1_manual.py` to a new file called `p3_2_cameras.py`.

The boilerplate already imports `HEAD_RGB_CAMERA` and `HEAD_DEPTH_CAMERA`. These camera objects provide a `get_frame()` method that returns the current image as a NumPy array.

Add code to:

1. Retrieve the RGB frame from `HEAD_RGB_CAMERA`
2. Retrieve the depth frame from `HEAD_DEPTH_CAMERA`
3. Display the RGB frame in a window called "RGB Camera"
4. Display the depth frame in a window called "Depth Camera"

Use OpenCV's `cv2.imshow(window_name, image)` function to display images. Place this code inside the main loop, before or after sending velocities—either works.

Expected Result

Run your script:

```
uv run p3_2_cameras.py
```

You should see two windows open:

- **RGB Camera**: showing the color view from the robot's head
- **Depth Camera**: showing depth information (closer objects appear brighter)

Manual control should still work—you can pan/tilt the head and see the camera views update in real time.

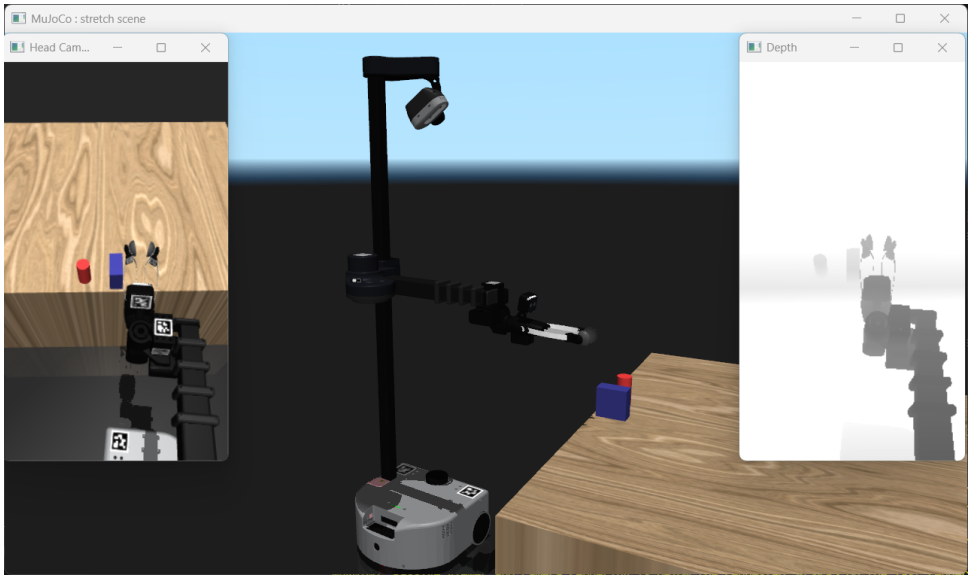


Figure 1: RGB and depth camera windows displaying simultaneously

Part 3 - Color Detection Mask

To track colored objects, you need to segment the image based on color. In this part, you'll create a binary mask that isolates blue objects in the camera view.

Objective

Add color-based segmentation to visualize which pixels in the RGB image match your target color (blue). This mask will be used in Part 4 to locate the object's position.

Task

Copy `p3_2_cameras.py` to a new file called `p3_3_mask.py`.

Color detection in computer vision typically uses the HSV (Hue, Saturation, Value) color space rather than RGB. HSV separates color information (hue) from brightness (value), making it more robust for color-based segmentation.

Add code to:

1. Convert the RGB frame to HSV using `cv2.cvtColor(rgb_frame, cv2.COLOR_BGR2HSV)`
2. Define lower and upper HSV bounds for blue. For example:

```
lower_blue = np.array([110, 100, 100])
upper_blue = np.array([130, 255, 255])
```

3. Create a binary mask using `cv2.inRange(hsv_frame, lower_blue, upper_blue)`
4. Display the mask in a third window called "Blue Mask"

The `cv2.inRange()` function returns a binary image where pixels within the HSV range are white (255) and all others are black (0).

Note on HSV values: Hue ranges from 0-179 in OpenCV (not 0-360). Blue is typically around 110-130. Saturation and Value range from 0-255. Adjust these ranges if your blue object isn't detected properly.

Expected Result

Run your script:

```
uv run p3_3_mask.py
```

You should see three windows:

- **RGB Camera**: the original color view
- **Depth Camera**: depth information
- **Blue Mask**: binary mask showing detected blue regions in white

Point the camera at a blue object—it should appear as a white blob in the mask window. Non-blue areas should be black.

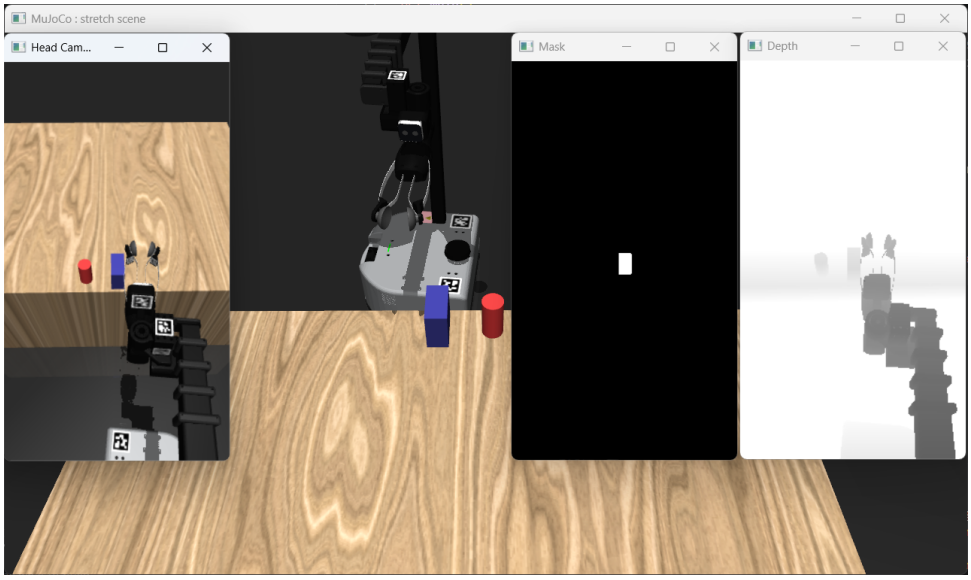


Figure 2: Three windows showing RGB camera, depth camera, and blue object mask

Part 4 - Visual Servoing with Manual Override

Now you'll implement closed-loop visual servoing: the robot will automatically move its head to center the blue object in the camera view, while still allowing manual override.

Objective

Use the binary mask from Part 3 to compute the object's position, calculate control commands to center it, and blend autonomous servoing with manual control using proportional blending.

Task

Copy `p3_3_mask.py` to a new file called `p3_4_servo.py`.

Visual servoing requires three steps: detect the object, compute the error, and apply proportional control.

Step 1: Find the Object's Centroid

Use `cv2.findContours()` to find contours (connected white regions) in the binary mask. If contours are found, select the largest one and compute its centroid using `cv2.moments()`.

The centroid gives you the (x, y) pixel coordinates of the object's center. Image coordinates are typically (0,0) at the top-left corner.

```
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
                               cv2.CHAIN_APPROX_SIMPLE)
if contours:
    largest_contour = max(contours, key=cv2.contourArea)
    M = cv2.moments(largest_contour)
    if M["m00"] > 0:
        cx = int(M["m10"] / M["m00"])
        cy = int(M["m01"] / M["m00"])
```

Step 2: Calculate Visual Error

Compute the error between the object's centroid and the image center. Get the image dimensions using `rgb_frame.shape`, then calculate:

```
height, width = rgb_frame.shape[:2]
error_x = cx - width // 2
error_y = cy - height // 2
```

Step 3: Proportional Control

Apply proportional control to generate servo velocities. The head pan should respond to horizontal error (error_x) and head tilt to vertical error (error_y). Use a gain constant (e.g., $K_p = 0.5$) to scale the error to appropriate velocity:

```
Kp = 0.5
servo_velocities = {
    "head_pan": -Kp * error_x / width,
    "head_tilt": Kp * error_y / height
}
```

Note the negative sign on pan—when the object is to the right (positive error_x), the robot should pan right (positive velocity), but due to coordinate conventions this requires negation.

Step 4: Blend with Manual Control

Don't replace the teleop velocities directly—blend them using `merge_proportional()`. This function from `stretch_toolkit` combines manual and autonomous commands, giving priority to manual control when the operator provides input:

```
teleop_velocities = teleop.get_normalized_velocities()
velocities = merge_proportional(teleop_velocities, servo_velocities)
```

Expected Result

Run your script:

```
uv run p3_4_servo.py
```

The robot should automatically pan and tilt its head to keep the blue object centered in the camera view. You can still manually control the robot—when you move the head, the autonomous servoing remains active but yields to your commands.

Try these tests:

- Let the robot track the object autonomously
- Move the object around—the robot should follow it
- Manually override by moving the head—control should be smooth
- Release the manual controls—servoing should resume

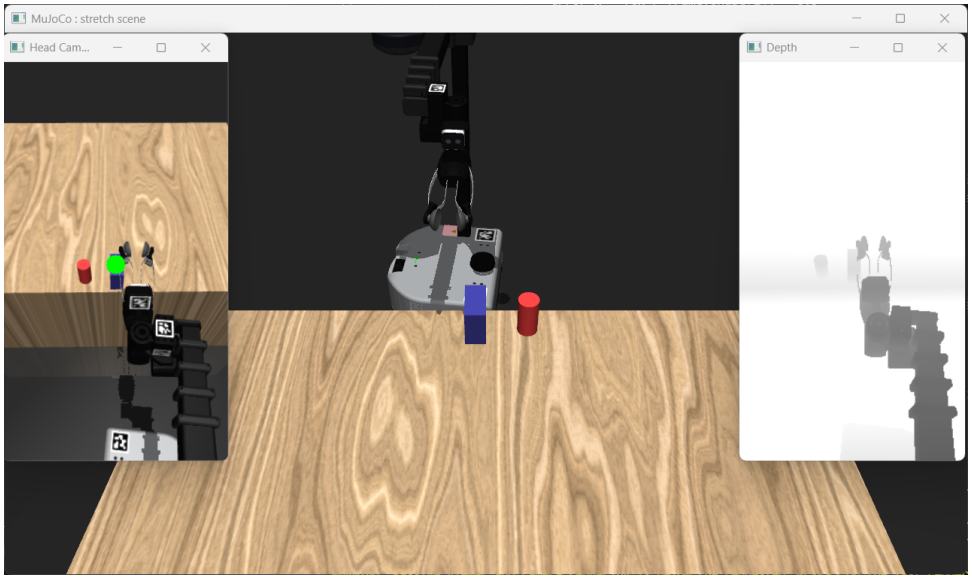


Figure 3: Robot performing visual servoing with object centered in view

Optional: Distance Measurement

Once you have the object's centroid coordinates, you can measure the distance to it using the depth camera. The `HEAD_CAMERA` object provides a `get_depth(x, y)` method that returns the distance in meters to the pixel at coordinates (x, y).

```
distance = HEAD_CAMERA.get_depth(cx, cy)
print(f"Distance to object: {distance:.2f}m")
```

You can also display the distance on the RGB frame using `cv2.putText()`:

```
cv2.putText(rgb_frame, f"Distance: {distance:.2f}m",
            (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
            1, (0, 255, 0), 2)
```

This provides real-time feedback about how far away the tracked object is, which is useful for approach behaviors or obstacle avoidance.

Deliverable

Submit a screen recording with voiceover demonstrating your completed visual servoing system from Part 4.

Your video should show:

- All three windows visible (RGB Camera, Depth Camera, Blue Mask)
- The robot autonomously tracking a blue object
- Manual override working smoothly with autonomous control
- Optional: distance measurement displayed

In your voiceover, briefly explain how the system works—covering HSV segmentation, proportional control, and the `merge_proportional()` blending function.